A1. Primer componente y catálogo

1. En `lib/src/quantity_stepper.dart` crea un `StatefulWidget` llamado `QuantityStepper` que muestre en una fila un `IconButton` con `Icons.remove`, el valor actual (empieza en 0) y un `IconButton` con `Icons.add`.
2. Al pulsar + el valor sube 1; al pulsar − baja 1.
3. Expórtalo desde `lib/ui_kit_dam.dart`.
4. Añade en Widgetbook un componente `QuantityStepper` con el caso de uso "Por defecto".
5. Responde en tu memoria: ¿por qué este componente es un `StatefulWidget` y `InfoBadge` no?

A2. Propiedades y valores por defecto

1. Añade las propiedades `min` (por defecto 0), `max` (por defecto 10), `step` (por defecto 1) e `initialValue` (por defecto, igual a `min`).
2. Valida con `assert` que `min < max`, que `step > 0` y que `initialValue` está entre `min` y `max`.
3. El valor nunca puede salir del rango, ni siquiera en modo *release*.
4. Desactiva el botón − en el mínimo y el botón + en el máximo.
5. Añade las propiedades de texto `semanticLabel` (por defecto `'Cantidad'`), `decrementTooltip` (`'Restar'`) e `incrementTooltip` (`'Sumar'`).
6. Añade en Widgetbook un caso "Configurable" con *knobs* para `min`, `max` y `step`.

A3. Eventos propios

1. Añade un evento `ValueChanged<int>? onChanged` que se emita con el nuevo valor cada vez que cambie.
2. Define un `enum StepperLimit { min, max }` y un `typedef LimitReached = void Function(StepperLimit limit);`.
3. Añade un evento `LimitReached? onLimitReached` que se emita cuando el valor llegue al mínimo o al máximo.
4. Ningún evento se emite si el valor no cambia.
5. En una pantalla de prueba, muestra un `SnackBar` con el texto "Has llegado al máximo" al alcanzar el límite superior.

A4. Pruebas

1. Saca el cálculo del siguiente valor a una función pura `nextStepperValue({required int current, required int delta, required int min, required int max})` y escribe al menos cuatro pruebas unitarias, incluidos los dos límites.
2. Escribe al menos tres pruebas de widget: que sumar muestra el nuevo valor y emite `onChanged`; que el botón − está desactivado en el mínimo; y que `onLimitReached` recibe `StepperLimit.max`.
3. Escribe una prueba *golden* del componente con valor 5.
4. Ejecuta `flutter test --coverage` e incluye en tu memoria una captura del porcentaje de cobertura de `quantity_stepper.dart`.

A5. Documentación

1. Documenta con `///` la clase, todas las propiedades públicas, los eventos, el `enum` y el `typedef`. La documentación de la clase incluye un ejemplo de uso.
2. Activa la regla `public_member_api_docs` y deja `flutter analyze` sin avisos.
3. Genera la documentación con `dart doc` y añade a tu memoria una captura de la página de `QuantityStepper`.
4. Actualiza el `README.md` del paquete con una sección para el nuevo componente.

A6. Empaquetado

1. Sube la versión del paquete a `1.1.0` en `pubspec.yaml` y explica en `CHANGELOG.md` qué se ha añadido.
2. Sube el paquete a un repositorio Git y crea la etiqueta `v1.1.0`.
3. Crea una aplicación nueva `stepper_demo` que dependa del paquete **por Git**, fijando la etiqueta.
4. Ejecuta `dart pub publish --dry-run` en el paquete y corrige los avisos que puedas. Anota en la memoria los que queden y por qué.

A7. Aplicación internacionalizada

1. Configura la internacionalización de `stepper_demo` en español (plantilla) e inglés.
2. Crea una pantalla de compra de entradas con un `QuantityStepper` (mínimo 1, máximo 8) y un `StarRating` para valorar el evento.
3. Muestra traducidos: el título, el número de entradas con plural ("1 entrada" / "3 entradas"), el precio total con formato de moneda (4,50 € por entrada) y la fecha del evento.
4. Pasa a los componentes los textos traducidos de `semanticLabel` y de los *tooltips*.
5. Cambia el idioma del dispositivo y adjunta capturas en ambos idiomas